

GUIDE TO USING SCRCPY AND ADB

Mirroring, Interaction, and Remote Administration

Technical Documentation

May 2026

GUIDE TO USING SCRCPY AND ADB FOR ANDROID SCREEN MIRRORING AND INTERACTION ON LINUX

Introduction

This guide provides step-by-step instructions on using **scrcpy** and **Android Debug Bridge (ADB)** on Linux to view and control your Android device's screen on your PC. It covers both wired and wireless configurations, switching modes, and detecting connected devices.

Prerequisites

- Linux System Requirements:**
 - A Linux-based operating system (Ubuntu, Fedora, etc.).
 - Ensure your system has access to a terminal.
- Install Required Tools:**
 - **ADB:** `bash sudo apt update sudo apt install adb`
 - **scrcpy:** `bash sudo apt install scrcpy`
- Android Device Configuration:**
 - **Enable Developer Options:**
 1. Go to **Settings > About phone**.
 2. Tap **Build number** seven times to unlock Developer Options.
 - **Enable USB Debugging:**
 1. Navigate to **Settings > System > Developer options**.

2. Toggle on **USB debugging**.

Wired Connection Setup

1. **Connect Android Device** to PC via USB Cable.
2. **Authorize ADB Connection:** On your Android device, allow USB debugging when prompted.
3. **Verify Device Detection:**

```
adb devices
```

You should see a list of connected devices with their unique IDs.

4. **Start scrcpy:**

```
scrcpy
```

This launches the Android screen mirroring window.

Wireless Connection Setup (Legacy)

1. **Prepare Android Device and PC:** Ensure your Android device and PC are on the same Wi-Fi network.
2. **Connect via USB Temporarily (First Time Only):**

```
adb tcpip 5555
```

This enables ADB over the network.

3. **Find Your Device's IP Address:** On Android, go to **Settings > About phone > Status > IP address**.
4. **Connect Wirelessly:**

```
adb connect <device_ip>:5555
```

5. **Verify Wireless Connection:**
-

```
adb devices
```

Your device should appear in the list.

6. Launch scrcpy:

```
scrcpy -e
```

The `-e` flag specifies the most recently connected wireless device.

Switching Between Wired and Wireless Modes

- **Switch to Wired Mode:** `bash scrcpy -d` The `-d` flag selects the device connected via USB.
- **Switch to Wireless Mode:** `bash scrcpy -e` The `-e` flag selects the most recently connected wireless device.

Detecting Connected Devices

- **View Connected Devices:** `bash adb devices` Lists devices with their connection type (e.g., USB or IP).
- **Monitor Device Connections in Real-Time:** `bash watch adb devices` Continuously updates the list of connected devices.

Advanced Options (Original)

1. **Specify Resolution:** `bash scrcpy --max-size 1024` Limits the resolution to reduce latency.
 2. **Record Screen:** `bash scrcpy --record file.mp4`
 3. **Forward Audio (Separate Tool Required):** Use `sndcpy` for audio forwarding.
-

Troubleshooting

1. Device Not Detected:

- Ensure USB debugging is enabled.
- Try reconnecting the USB cable or restarting ADB:

```
adb kill-server && adb start-server
```

2. Wireless Connection Issues:

- Ensure both devices are on the same network.
- Verify the IP address and port.

Migrating ADB Authorization & Identity (Full Technical Guide)

To ensure your phone continues to trust your computer after you reinstall your Linux OS, you need to back up and restore your ADB (Android Debug Bridge) keys. **scrcpy** relies entirely on ADB for the connection and authorization.

Files to Export

All pairing and authorization data is stored in the hidden `.android` directory in your home folder. You need to copy the following files: 1. `~/.android/adbkey` : Your computer's private RSA key. 2. `~/.android/adbkey.pub` : Your computer's public RSA key. 3. `~/.android/adb_known_hosts.pb` (if it exists): This file stores specific pairing data for Android 11+ Wireless Debugging.

Steps to Migrate

1. **On your current OS:** Copy the files listed above to a USB drive or cloud storage.
2. **On the new OS:**
 - Install `scrcpy` and `adb`.
 - Run `adb devices` once to ensure the `.android` directory is created (it will generate new keys by default).
 - Replace the newly generated files in `~/.android/` with the ones you backed up.
3. **Set Correct Permissions (Critical):** ADB is modeled after SSH-style private key

handling. If permissions are too open (like `777`), ADB may ignore or reject the keys as insecure. Set sane permissions:

```
chmod 700 ~/.android
chmod 600 ~/.android/adbkey
chmod 644 ~/.android/adbkey.pub
```

4. Restart the ADB server:

```
adb kill-server && adb start-server
```

Why Permissions Matter

A private key (like `adbkey`) readable by everyone is considered invalid/insecure. While `adbkey.pub` can be generally readable, the private `adbkey` must be restricted to your user. Restoring with `777` permissions (readable/writable/executable by everyone) is a common cause for migration failure.

Why this works

The RSA public key in `adbkey.pub` is what the phone uses to identify your computer. By restoring both the keys and the correct permissions, your computer “identifies” itself with the exact same trusted signature, and your phone will allow the connection immediately without a new prompt.

Note: If you are using `scrcpy` over a wireless connection, you may still need to run `adb connect <phone_ip>:5555` once on the new OS to re-establish the network link, but it will not ask for authorization on the phone screen.

NEW: MODERN WIRELESS PAIRING (ANDROID 11+)

Benefit: This method is entirely wireless from start to finish. You do **not** need to physically plug in the phone to establish the initial connection.

1. Enable Wireless Debugging:

- On Android: Go to **Settings > Developer options**.
- Toggle on **Wireless debugging**.
- Tap the “Wireless debugging” text to enter its sub-menu.

2. Enter Pairing Mode:

- Select “**Pair device with pairing code**”.
- You will see an **IP address & Port** (e.g., `192.168.1.50:37123`) and a **Pairing code**.

3. Pair from Linux Terminal:

```
adb pair <ip_address>:<pairing_port>
```

- Type the **Pairing code** when prompted.

4. Final Connection:

- The pairing window will close. Look at the main “Wireless debugging” screen for the **current IP address and Port** (Note: The connection port is usually different from the pairing port).

```
adb connect <ip_address>:<connection_port>
```

5. Launch scrcpy:

```
scrcpy
```

ADB & SCRCPY MASTER CHEAT SHEET

1. Multi-Device Management

- List devices with details: `adb devices -l`
- Target a specific device: `adb -s <serial_number> <command>`
 - Example: `adb -s 192.168.1.50:5555 shell`

2. File System Management

- Enter Interactive Shell: `adb shell`
- List files (detailed): `adb shell ls -la /sdcard/`
- Find a file/folder: `adb shell find /sdcard/ -name "*search_term*"`
- Check Disk Usage: `adb shell df -h`
- Push (Upload) a file: `adb push local_path /sdcard/remote_path`
- Pull (Download) a file: `adb pull /sdcard/remote_path local_path`

3. Application Administration

- Install APK: `adb install app.apk`
- Reinstall/Update (keep data): `adb install -r app.apk`
- Uninstall App: `adb uninstall com.package.name`
- List ALL packages: `adb shell pm list packages`
- List 3rd Party (User) apps: `adb shell pm list packages -3`
- List System apps: `adb shell pm list packages -s`
- Clear App Data (Reset): `adb shell pm clear com.package.name`
- Find APK path on device: `adb shell pm path com.package.name`

4. Execution & Remote Control

- Start an App (Launcher): `adb shell monkey -p com.package.name -c android.intent.category.LAUNCHER 1`
- Force Stop an App: `adb shell am force-stop com.package.name`
- Open URL in Browser: `adb shell am start -a android.intent.action.VIEW -d "https://example.com"`
- Take Screenshot: `adb shell screencap -p /sdcard/screen.png && adb pull /sdcard/screen.png`
- Record Screen: `adb shell screenrecord /sdcard/demo.mp4` (Ctrl+C to stop)
- Type Text: `adb shell input text "Hello_World"`
- Key Events:
 - Home: `adb shell input keyevent 3`
 - Back: `adb shell input keyevent 4`
 - Power: `adb shell input keyevent 26`

5. scrcpy (Remote Vision & Interaction)

- **Standard Start:** `scrcpy`
- **Target specific device:** `scrcpy -s <serial>`
- **Wireless scrcpy:** `scrcpy --tcpip=192.168.1.50:5555`
- **Record while mirroring:** `scrcpy --record file.mp4`
- **Turn device screen OFF:** `scrcpy --turn-screen-off` (Shortcut: Mod+o)
- **Prevent device sleep:** `scrcpy --stay-awake`
- **Low Latency Settings:** `scrcpy --max-size=1024 --video-bit-rate=2M --max-fps=30`

6. Wireless Setup & Logs

- **Enable Wireless (USB required first):** `adb tcpip 5555`
 - **Connect via IP:** `adb connect <IP_ADDRESS>:5555`
 - **View Live Logs (filtered by app):** `adb logcat --pid=$(adb shell pidof -s com.package.name)`
 - **Device Battery Stats:** `adb shell dumpsys battery`
 - **Reboot:** `adb reboot`
-

Conclusion

Using **scrcpy** and **ADB**, you can seamlessly view and control your Android device from your Linux PC. Whether using a wired connection for stability or a wireless connection for convenience, these tools offer flexibility and powerful features to enhance your workflow.